

LangChain Debugging — Worked Example: Solution

[Back to the exercise](#) · [Round 1: Basic](#) · [Round 2: Intermediate](#) · [Round 3: Real-world](#) · [Round 4: Multi-agent](#) · [Hints](#)

- [Round 1: Basic](#)
- [Round 2: Intermediate](#)
- [Round 3: Real-world](#)
- [Round 4: Multi-agent](#)
- [Real cause vs. symptom fix](#)

[Back to the exercise](#) · [Round 1: Basic](#) · [Round 2: Intermediate](#) · [Round 3: Real-world](#) · [Round 4: Multi-agent](#) · [Hints](#)

Round: Basic

Real cause: the prompt template declares one variable, `{question}`, but the caller invoked the chain with a dict keyed `"input"` instead. LangChain's error message actually names the mismatch directly (`Expected: ['question'] Received: ['input']`) — it's easy to miss because the error type, `KeyError`, makes it feel like a Python dictionary bug rather than a straightforward naming mismatch.

The fix:

```
result = chain.invoke({"question": "What is the refund policy?"})
```

The dictionary key passed to `.invoke()` has to match the template's variable name exactly.

Test that would have caught it:

```
def test_chain_invoke_uses_the_correct_input_key():
    result = chain.invoke({"question": "What is the refund policy?"})
    assert isinstance(result, str)
    assert len(result) > 0
```

The error message here already tells you almost everything — reading it fully, instead of jumping straight to "must be an API problem" because it involves LangChain, is most of the work.

[Back to the exercise](#) · [Round 1: Basic](#) · [Round 2: Intermediate](#) · [Round 3: Real-world](#) · [Round 4: Multi-agent](#) · [Hints](#)

Round: Intermediate

Real cause: the model did exactly what a chat model commonly does — it wrapped its JSON answer in a markdown code fence, because that's a normal, helpful way to present JSON to a person reading it in a chat UI. `PydanticOutputParser`'s default parsing doesn't strip that fence before trying to load the JSON, so it sees the fence markers as part of the text and fails. This is a chain-layer bug (the parser's own handling), not an API-layer bug — the API call itself succeeded and returned a perfectly reasonable answer.

The fix:

```
from langchain_core.output_parsers import PydanticOutputParser

parser = PydanticOutputParser(pydantic_object=Answer)
chain = prompt | ChatOpenAI() | parser
```

The maintained fix is to instruct the model explicitly, inside the prompt template, not to use markdown fences (`parser.get_format_instructions()` already generates wording for this — include it in the prompt), and to use a parser version that strips a leading/trailing fence before parsing, since even a well-instructed model still does this occasionally.

Test that would have caught it:

```
def test_parser_handles_markdown_fenced_json():
    fenced_reply = "`json\n{\"summary\": \"Refunds within 30 days.\", \"confide\nresult = parser.parse(fenced_reply)
assert result.summary == \"Refunds within 30 days.\""
```

Testing the parser directly against a fenced string, without making a real API call, is the fast way to catch this — the bug is entirely in the parser's own handling of a shape that real models produce constantly.

[Back to the exercise](#) · [Round 1: Basic](#) · [Round 2: Intermediate](#) · [Round 3: Real-world](#) · [Round 4: Multi-agent](#) · [Hints](#)

Round: Real-world

Real cause: LCEL's prompt templates use `{...}` as their own variable syntax. Any literal `{` or `}` inside the *value* being substituted in (not the template itself) gets misread as another variable placeholder the template is supposed to fill — which it can't, because nothing supplied a value named `status`. This only fails for the specific inputs that happen to contain braces, which is exactly why it's invisible until real, varied user input starts flowing through.

The fix:

```
def escape_braces(text):
    escaped = text.replace("{", "{{")
    escaped = escaped.replace("}", "}}")
    return escaped

safe_question = escape_braces(user_question)
result = chain.invoke({"question": safe_question})
```

Doubling every literal brace (`{{` / `}}`) is the standard escape LCEL's templates understand — it tells the template engine "treat this brace as a literal character, not a variable marker."

Test that would have caught it:

```
def test_chain_handles_question_containing_curly_braces():
    tricky_question = 'Why does my config return {"status": "error"} instead of 2
    result = chain.invoke({"question": escape_braces(tricky_question)})
    assert isinstance(result, str)
```

A test suite built only from clean, hand-typed questions will never exercise this path — the test has to deliberately include a question with braces in it, the same way real user input eventually will.

[Back to the exercise](#) · [Round 1: Basic](#) · [Round 2: Intermediate](#) · [Round 3: Real-world](#) · [Round 4: Multi-agent](#) · [Hints](#)

Round: Multi-agent

Real cause: the same brace-collision bug as Round 3, but the source of the unsafe text is now another agent's output, not direct user input — so the fix from Round 3, applied only where direct user text enters the system, never covers it. Any text handed into an LCEL template has this problem, no matter which agent (or human) produced it.

The fix:

```
def verify_claim(state):
    safe_findings = escape_braces(state["research_findings"])
    result = verification_chain.invoke({"claim_context": safe_findings})
    return Command(goto="writer_agent", update={"analysis": result})
```

The general fix is to escape braces at every boundary where free-form text (from a user, from a tool result, from another agent's output) enters an LCEL template — not just the one boundary that happened to be tested first.

Test that would have caught it:

```
def test_analysis_agent_handles_findings_containing_json_snippets(fake_research_a
    fake_findings = 'the API returned {"error": "rate_limited"} on the first atte
    state = {"research_findings": fake_findings}
    result = analysis_agent_node(state)
    assert result.update["analysis"] is not None
```

Testing the Analysis agent alone still catches this, as long as the test feeds it findings shaped like what Research realistically produces (including an occasional JSON snippet) — the bug doesn't actually require the full 4-agent pipeline to reproduce, it just wasn't found because nobody tested that specific shape of input before it happened in production.

[Back to the exercise](#) · [Round 1: Basic](#) · [Round 2: Intermediate](#) · [Round 3: Real-world](#) · [Round 4: Multi-agent](#) · [Hints](#)

Real cause vs. symptom fix

A symptom fix here would be catching the `KeyError` from Round 1 and silently falling back to a generic answer, stripping markdown fences with a one-off string replace scattered at the one call site that happened to break (Round 2), or wrapping just the one input field that crashed in a `try/except` instead of understanding that *any* free-form text hitting an LCEL template has the same brace problem (Rounds 3 and 4). The real-cause fixes above all target the actual mechanism — LCEL's `{...}` template syntax colliding with literal braces in real content — which is why the same `escape_braces()` helper fixes both Round 3 and Round 4 instead of needing a new patch for every place free-form text happens to enter a chain. That reuse is a good sign you found the real cause, not a symptom: a real fix tends to generalize; a symptom fix tends to need reapplying every time the same bug shows up somewhere new.